Answer Sheet – Unit 1 & 2 (20 Marks Each)

Module 1: Introduction to HDFS and Hive

Q1. (Understand – L2)

Explain the overview of Hadoop, including its core principles and how it addresses the challenges of big data storage and processing.

Include a real-world scenario where a global e-commerce company uses Hadoop to manage petabytes of daily transaction logs.

Introduction

Hadoop is an open-source, Java-based framework developed by Apache Software Foundation, designed for distributed storage and parallel processing of massive datasets across clusters of commodity (off-the-shelf) hardware. It was inspired by Google's MapReduce paper and the Google File System (GFS), and has since become the foundation of the Big Data ecosystem.

Core Challenges of Big Data That Hadoop Addresses

Before Hadoop, organizations faced significant challenges:

- **Volume**: Storing terabytes to petabytes of data was expensive using traditional RDBMS.
- **Velocity**: Processing real-time or near-real-time data streams was not feasible.
- **Variety**: Structured, semi-structured, and unstructured data could not be handled uniformly.
- **Fault Tolerance**: A single node failure could bring down entire systems.

Hadoop addresses all of these through its distributed, fault-tolerant design.

Core Principles of Hadoop

1. **Data Locality**: Instead of moving data to computation, Hadoop moves computation to where data resides. This minimizes expensive network transfers.
2. **Fault Tolerance**: Data is replicated (by default, 3 copies) across different nodes. If one node fails, the system automatically uses another replica.
3. **Horizontal Scalability**: New nodes can be added to the cluster without downtime, allowing seamless scaling.

4. **Commodity Hardware**: Hadoop runs on low-cost, general-purpose hardware, making it cost-effective.
5. **Distributed Processing**: Jobs are split and executed in parallel across multiple nodes.

Core Components of Hadoop

Component	Role
HDFS (Hadoop Distributed File System)	Distributed storage layer
MapReduce	Batch processing framework (Hadoop 1.x)
YARN	Resource management and job scheduling (Hadoop 2.x+)
Hadoop Common	Shared utilities and libraries

Hadoop Ecosystem Tools

Beyond the core, the Hadoop ecosystem includes:

- **Hive** – SQL-like querying over HDFS data
- **HBase** – NoSQL column-oriented database on HDFS
- **Pig** – Data flow scripting language
- **Sqoop** – Data transfer between HDFS and RDBMS
- **Flume** – Log data ingestion
- **Oozie** – Workflow scheduler

Real-World Scenario: Global E-Commerce Company

Consider **FlipMart**, a global e-commerce company generating 10 TB of transaction logs daily — including purchase records, user clickstreams, cart events, and payment logs.

Without Hadoop:

- Traditional databases cannot store or query such volumes efficiently.
- Replication and failover mechanisms are costly.
- Batch analytics (e.g., daily sales reports) would take hours or days.

With Hadoop:

- Transaction logs are ingested via **Flume** and stored in **HDFS** across 500 nodes.
- HDFS replicates each file 3 times — ensuring zero data loss even if 2 nodes fail simultaneously.
- **MapReduce** / **Spark** jobs compute daily revenue summaries, top-selling products, and regional sales breakdowns in minutes.
- **Hive** allows business analysts to run SQL queries like `SELECT product_id, SUM(revenue) FROM transactions GROUP BY product_id` without writing any Java code.
- Over time, historical logs (petabytes) remain queryable and searchable without archiving or deleting old data.

Advantages of Hadoop

- Open-source and free to use
- Processes both structured and unstructured data
- Highly fault-tolerant and reliable
- Linear scalability – add more nodes as data grows
- Supports diverse workloads: batch, interactive, real-time (with Spark)

Limitations of Hadoop

- Not suitable for small datasets (overhead is high)
- MapReduce is slow for iterative algorithms (compared to Spark)
- Difficult to install, configure, and manage
- No support for real-time streaming natively (requires Kafka + Spark)

Conclusion

Hadoop revolutionized how organizations handle Big Data by providing a scalable, fault-tolerant, and cost-effective platform. For companies like global e-commerce platforms dealing with petabytes of transactional data, Hadoop provides the backbone to store, process, and derive insights from data that would be otherwise unmanageable using conventional systems.

Q2. (Apply – L3) Demonstrate the HDFS architecture with a diagram and apply it to a scenario where data is distributed across multiple nodes for high availability. Consider a banking organization storing sensitive customer data with strict fault-tolerance requirements.

Introduction to HDFS

HDFS (Hadoop Distributed File System) is the primary storage component of Hadoop. It is designed to store very large files reliably across a cluster of machines and to provide high-throughput access to application data.

Design Goals of HDFS

- Handle hardware failure automatically
- Stream access to large datasets
- Store very large files (GBs to TBs)
- Work on commodity hardware
- Portability across heterogeneous hardware and software platforms

HDFS Architecture

HDFS follows a **Master-Slave architecture** consisting of:

```
                    +------------------+
                    |    NameNode      |
                    |  (Master Node)   |
                    |  - Metadata      |
                    |  - File Tree     |
```

```
                      |  - Block Map     |
                      +--------+---------+
                               |
          +-------------------+--------------------+
          |                   |                    |
+--------+-------+  +--------+-------+  +---------+------+
|   DataNode 1   |  |   DataNode 2   |  |   DataNode 3   |
|  Block A (1)   |  |  Block A (2)   |  |  Block A (3)   |
|  Block B (2)   |  |  Block C (1)   |  |  Block B (3)   |
+----------------+  +----------------+  +----------------+
```

1. NameNode (Master)

- Manages the **filesystem namespace** (directory tree, file-to-block mapping)
- Stores metadata in **FsImage** (filesystem snapshot) and **EditLog** (recent changes)
- Does NOT store actual data — only block locations
- Single point of failure in older versions; addressed by **Secondary NameNode** and **HA NameNode** in newer versions

2. DataNodes (Slaves)

- Store actual data in the form of **blocks** (default block size: 128 MB)
- Periodically send **heartbeats** and **block reports** to the NameNode
- Perform data read/write operations as directed by the NameNode or client

3. Secondary NameNode

- Not a backup NameNode — it periodically merges FsImage and EditLog to prevent EditLog from growing too large
- In Hadoop HA setups, a **Standby NameNode** replaces the Secondary NameNode for true failover

File Storage in HDFS

When a file is stored:

1. The file is split into **blocks** (e.g., a 512 MB file → 4 blocks of 128 MB each)
2. Each block is **replicated 3 times** (default replication factor)
3. Replicas are placed on **different nodes** (and preferably different racks) using the **Rack-Aware Replication Policy**:
    - 1st replica: local node
    - 2nd replica: different node on same rack
    - 3rd replica: node on a different rack

HDFS Rack-Aware Placement

```
Rack 1                        Rack 2
+-----------+  +-----------+  +-----------+  +-----------+
| DataNode1 |  | DataNode2 |  | DataNode3 |  | DataNode4 |
```

```
| Block A(1)|  | Block A(2)|  | Block A(3)|  |           |
+-----------+  +-----------+  +-----------+  +-----------+
```

This ensures: if an entire rack goes down, at least one replica remains accessible on the other rack.

Real-World Scenario: Banking Organization

SecureBank stores sensitive customer data: account details, transaction histories, KYC documents, and loan records totalling 50 TB, with strict requirements for:

- **Zero data loss** (even during hardware failure)
- **Continuous availability** (24/7 uptime)
- **Regulatory compliance** (data must be retained for 7 years)

HDFS Implementation at SecureBank:

1. **Storage**: All customer data is stored as encrypted files in HDFS across 100 DataNodes.
2. **Replication Factor = 3**: Every data block is replicated on 3 different nodes across 2 racks.
3. **HA NameNode Setup**: Primary NameNode + Standby NameNode with **ZooKeeper** for automatic failover. If the active NameNode crashes, ZooKeeper promotes the Standby NameNode within seconds.
4. **Heartbeat Monitoring**: If a DataNode stops sending heartbeats for 10 minutes, the NameNode marks it as dead and automatically re-replicates its blocks onto healthy nodes.
5. **HDFS Snapshots**: Daily snapshots of the HDFS namespace are taken for point-in-time recovery, satisfying regulatory requirements.

HDFS vs Traditional File Systems

Feature	HDFS	Traditional FS
Scale	Petabytes	Gigabytes–Terabytes
Fault Tolerance	Built-in replication	External RAID
Hardware	Commodity	High-end servers
Access Pattern	Streaming reads	Random reads
Modification	Write-once, append-only	Full random write

Conclusion

HDFS's master-slave architecture, block-based storage, rack-aware replication, and automatic failure recovery make it an ideal choice for organizations like banks that demand high availability, fault tolerance, and scalable storage for sensitive, mission-critical data.

Q3. (Analyze – L4) Examine the YARN architecture in Hadoop and analyze how it manages resources differently from earlier versions. Use the scenario of a multi-tenant cloud environment running simultaneous batch and interactive workloads to highlight the differences.

Introduction to YARN

YARN (Yet Another Resource Negotiator), introduced in **Hadoop 2.0**, is the resource management layer of Hadoop. It decouples resource management from job execution, addressing the fundamental limitations of the original Hadoop 1.x architecture.

Hadoop 1.x Architecture (Pre-YARN)

In Hadoop 1.x, the **JobTracker** was responsible for:

- Cluster resource management
- Job scheduling
- Task monitoring and fault recovery

This caused severe problems:

- **Single Point of Failure**: If JobTracker crashed, all running jobs were lost.
- **Scalability Bottleneck**: JobTracker could manage only ~4,000 nodes and ~40,000 concurrent tasks.
- **Support for Only MapReduce**: No other processing frameworks (Spark, Tez, MPI) could run natively.
- **Poor Resource Utilization**: Resources were allocated as fixed slots (Map slots, Reduce slots), leading to wastage.

YARN Architecture

YARN introduces a clear separation of concerns:

```
+-------------------------------------------------------+
|                   ResourceManager                     |
|  +-----------------+   +-------------------------+   |
|  | Scheduler       |   | ApplicationsManager     |   |
|  | (Resource alloc)|   | (App lifecycle mgmt)    |   |
|  +-----------------+   +-------------------------+   |
+-------------------------------------------------------+
        |                        |
+-------+--------+       +-------+--------+
| NodeManager 1  |       | NodeManager 2  |
| - Resource mon.|       | - Resource mon.|
| - Container    |       | - Container    |
|   execution    |       |   execution    |
+-------+--------+       +-------+--------+
        |
+-------+--------+
| ApplicationMaster|
| (per application)|
| - Negotiate      |
|   resources      |
| - Monitor tasks  |
+------------------+
```

1. ResourceManager (RM)

The global master with two components:

- **Scheduler**: Allocates resources (CPU, memory) to running applications using pluggable scheduling policies (FIFO, Capacity Scheduler, Fair Scheduler). Does NOT monitor or restart tasks.
- **ApplicationsManager (AsM)**: Accepts job submissions, negotiates the first container for the ApplicationMaster, and restarts failed ApplicationMasters.

2. NodeManager (NM)

- Runs on every worker node
- Manages resources (CPU cores, memory, disk, network) on that node
- Launches and monitors **Containers** (units of resource allocation)
- Reports resource usage back to the ResourceManager

3. ApplicationMaster (AM)

- One per submitted application (e.g., one AM per Spark job, one per MapReduce job)
- Negotiates with ResourceManager for containers
- Works with NodeManagers to run and monitor application tasks
- Restarts failed tasks within its scope

4. Container

- A bundle of resources (vCores + memory) allocated on a node
- Every task runs inside a container
- Provides resource isolation between tasks

YARN Workflow

```
1. Client submits application to ResourceManager
2. RM launches ApplicationMaster in a container
3. AM registers with RM and requests resource containers
4. RM's Scheduler allocates containers on suitable NodeManagers
5. AM instructs NodeManagers to launch tasks in containers
6. Tasks execute, send progress to AM
7. AM reports completion to RM; resources are released
```

YARN vs Hadoop 1.x Comparison

Feature	Hadoop 1.x (JobTracker)	YARN
Resource Management	JobTracker	ResourceManager
Job Scheduling	JobTracker	ResourceManager Scheduler
Task Monitoring	TaskTracker	ApplicationMaster (per app)

Feature	Hadoop 1.x (JobTracker)	YARN
Fault Tolerance	Poor (JobTracker = SPOF)	RM HA + AM restart
Scalability	~4,000 nodes	10,000+ nodes
Framework Support	MapReduce only	MapReduce, Spark, Tez, MPI, etc.
Resource Model	Fixed slots	Dynamic containers

YARN Scheduling Policies

- **FIFO Scheduler**: Jobs run in submission order — simple but inefficient for mixed workloads.
- **Capacity Scheduler**: Cluster divided into queues (e.g., 60% batch, 40% interactive). Each queue has guaranteed capacity.
- **Fair Scheduler**: All jobs get equal share of resources over time; short jobs don't starve.

Real-World Scenario: Multi-Tenant Cloud Environment

CloudAnalytics Inc. runs a shared Hadoop cluster with 500 nodes for:

- **Team A (Data Engineers)**: Long-running nightly batch ETL jobs (8–10 hours)
- **Team B (Data Scientists)**: Interactive Spark ML model training (30–60 minutes)
- **Team C (BI Analysts)**: Quick Hive SQL queries (2–5 minutes)

YARN Configuration:

- **Capacity Scheduler** with 3 queues:
  - `batch-queue`: 50% capacity for ETL jobs
  - `ml-queue`: 30% capacity for ML training
  - `interactive-queue`: 20% capacity for BI queries with preemption enabled
- Each job runs with its own **ApplicationMaster**, so a crash in Team A's ETL job doesn't affect Team B's Spark job.
- **Preemption** is enabled on the `interactive-queue` so BI analysts get fast responses even when batch jobs are consuming most resources.
- YARN's RM HA with ZooKeeper ensures the cluster continues operating even if the primary RM fails.

Without YARN (Hadoop 1.x): All three teams would compete for fixed Map/Reduce slots via a single JobTracker, causing interactive queries to wait behind long-running batch jobs with no isolation.

Conclusion

YARN's separation of resource management (ResourceManager) from application logic (ApplicationMaster) enables Hadoop to scale massively, support diverse workloads simultaneously, and provide robust fault tolerance — making it the foundation of modern multi-tenant Big Data platforms.

Q4. (Evaluate – L5) Assess the anatomy of file read and write operations in Hadoop, evaluating their efficiency in handling large-scale data transfers. Consider a media streaming company that frequently uploads and streams high-definition video files to millions of users.

Introduction

Understanding how Hadoop handles file I/O is critical for evaluating its suitability for large-scale data systems. HDFS is optimized for **high-throughput sequential access** to large files, which aligns well with streaming workloads.

Anatomy of HDFS Write Operation

When a client writes a file to HDFS, the following steps occur:

```
Client → NameNode: "I want to write file X"
NameNode → Client: "Write block 1 to [DN1, DN2, DN3]"
Client → DN1 → DN2 → DN3: Data pipeline (streaming write)
DN3 → DN2 → DN1 → Client: Acknowledgment pipeline
(Repeat for each block)
Client → NameNode: "File write complete"
```

Detailed Steps:

1. **Client Request**: The client calls `create()` on the HDFS DistributedFileSystem API.
2. **NameNode Validation**: NameNode checks permissions, ensures file doesn't already exist, and records the new file in the namespace.
3. **Data Streaming**: The client writes data to a **DFSOutputStream** which splits data into **packets** (64 KB each).
4. **Pipeline Construction**: Packets are sent through a **write pipeline** — Client → DataNode1 → DataNode2 → DataNode3 (based on replication factor).
5. **Acknowledgment**: Each DataNode acknowledges receipt. The client advances only after receiving ACKs.
6. **Block Completion**: Once a full block is written (128 MB), the client requests a new block from the NameNode.
7. **Close**: Client calls `close()`. The NameNode is notified that the file write is complete.

Fault Handling During Write:

- If a DataNode in the pipeline fails, it is removed from the pipeline, the block is written to the remaining nodes, and the NameNode is informed to re-replicate the block later.

Anatomy of HDFS Read Operation

```
Client → NameNode: "Give me block locations for file X"
NameNode → Client: Block list + DataNode addresses (sorted by proximity)
Client → Nearest DataNode: "Give me Block 1"
DataNode → Client: Block data streamed
(Repeat for each block)
```

Detailed Steps:

1. **Open File**: Client calls `open()` on DistributedFileSystem.
2. **Get Block Locations**: NameNode returns metadata — list of blocks and their DataNode locations, sorted by **network proximity** to the client (rack-awareness).
3. **Read from Nearest DataNode**: Client connects directly to the nearest DataNode and reads block data via `DFSInputStream`.
4. **Checksum Verification**: HDFS verifies checksums on every block to detect corruption.
5. **Failover on Error**: If a DataNode fails during read, `DFSInputStream` transparently switches to the next closest replica.
6. **Close**: Stream is closed after all blocks are read.

Efficiency Analysis

Aspect	Write	Read
Throughput	High (pipeline parallelism)	Very High (direct DataNode read)
Latency	Moderate (ACK pipeline overhead)	Low (reads from nearest replica)
Fault Tolerance	Strong (automatic re-replication)	Strong (automatic failover)
Concurrency	Single writer per file	Multiple concurrent readers
Optimization	Large block size reduces metadata ops	Data locality reduces network I/O

Key Efficiency Factors:

- **Large Block Size (128 MB)**: Reduces the number of blocks, minimizing metadata lookups at the NameNode.
- **Pipeline Writes**: Data is streamed concurrently to all replicas without the client needing to send it 3 times separately.
- **Data Locality for Reads**: The scheduler places computation near data, eliminating expensive data transfers.
- **Sequential Access Optimization**: HDFS is tuned for streaming large files sequentially — not random access.

Real-World Scenario: Media Streaming Company

StreamVision uploads 1,000 new HD video files (each ~4 GB) daily and streams them to 50 million concurrent users.

Upload (Write) Workflow:

- Each 4 GB video is split into **32 blocks of 128 MB**.
- Each block is written via a 3-node pipeline, taking ~2 seconds per block on a 1 Gbps network.
- Total upload time per file: ~64 seconds (vs. minutes if written serially to a single disk).
- The 3x replication ensures no video is lost even during disk failures.

Streaming (Read) Workflow:

- When a user in Mumbai requests a video, the HDFS client (served by an edge DataNode in the Mumbai region) reads from the **nearest replica**.

- 50 million users can simultaneously stream different blocks of different files from different DataNodes — **massively parallel reads** with no bottleneck.
- Checksum verification ensures no corrupted frame is served to users.

Evaluation:

- HDFS write efficiency is excellent for bulk uploads — pipeline replication minimizes write amplification.
- HDFS read efficiency is outstanding for streaming — locality-aware, parallel, with automatic failover.
- Limitation: HDFS is **not suited for random seeks** (e.g., jumping to a timestamp mid-stream) because it's optimized for sequential access. For that, StreamVision would use an additional **HBase** or object storage layer for random-access metadata.

Conclusion

HDFS's pipelined write mechanism and locality-aware read mechanism make it exceptionally efficient for large-scale sequential data transfers. For a media streaming company dealing with petabytes of video data and millions of concurrent streams, HDFS provides the high-throughput, fault-tolerant storage backbone required at scale.

Q5. (Create – L6) Design a conceptual Hive database schema using appropriate data types, views, built-in operators, and functions to support complex analytical queries. Propose this schema for a healthcare analytics platform that needs to process patient records stored in HDFS while ensuring seamless interaction with the underlying Hadoop framework.

Introduction to Apache Hive

Apache Hive is a data warehousing infrastructure built on top of Hadoop. It provides an SQL-like query language called **HiveQL (HQL)** that gets translated into MapReduce or Tez jobs for execution on Hadoop clusters. Hive is ideal for batch analytical queries on large datasets stored in HDFS.

Hive Architecture

```
+-------------------------------------------+
|        User / BI Tool (JDBC/ODBC)         |
+-------------------------------------------+
              |
+-------------------------------------------+
|          Hive Driver                      |
|  - Compiler (HQL → Execution Plan)        |
|  - Optimizer (Logical → Physical Plan)    |
|  - Executor (submits to YARN)             |
+-------------------------------------------+
              |
+-------------------------------------------+
|          Metastore                        |
|  (Schema metadata: tables, columns,       |
|   partitions, SerDe info → MySQL/Derby)   |
```

```
+-----------------------------------------+
              |
+-----------------------------------------+
|          HDFS (Data Storage)            |
+-----------------------------------------+
```

Healthcare Analytics Platform Schema Design

Platform: HealthInsight Analytics **Goal**: Process de-identified patient records, diagnoses, prescriptions, and hospital visits to support population health analytics, readmission prediction, and drug efficacy research.

Database

```sql
CREATE DATABASE IF NOT EXISTS healthcare_db
COMMENT 'Healthcare Analytics Platform Database'
LOCATION '/user/hive/warehouse/healthcare_db.db';

USE healthcare_db;
```

Table 1: Patients

```sql
CREATE TABLE IF NOT EXISTS patients (
    patient_id      BIGINT,
    first_name      STRING,
    last_name       STRING,
    dob             DATE,
    gender          STRING,
    blood_group     STRING,
    contact_number  STRING,
    city            STRING,
    state           STRING,
    zip_code        STRING,
    insurance_id    STRING
)
COMMENT 'Master patient demographic records'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS ORC
TBLPROPERTIES ('orc.compress'='SNAPPY');
```

Table 2: Hospital Visits (Partitioned)

```sql
CREATE TABLE IF NOT EXISTS hospital_visits (
    visit_id        BIGINT,
    patient_id      BIGINT,
    hospital_id     INT,
    doctor_id       INT,
    visit_type      STRING,     -- 'OPD', 'IPD', 'Emergency'
    admission_date  TIMESTAMP,
    discharge_date  TIMESTAMP,
    diagnosis_code  STRING,     -- ICD-10 code
    severity        STRING,     -- 'Low', 'Moderate', 'High', 'Critical'
    bill_amount     DECIMAL(12,2),
    readmitted      BOOLEAN
)
COMMENT 'Patient hospital visit records partitioned by year and month'
PARTITIONED BY (visit_year INT, visit_month INT)
CLUSTERED BY (patient_id) INTO 32 BUCKETS
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS ORC
TBLPROPERTIES ('orc.compress'='SNAPPY', 'transactional'='true');
```

Table 3: Prescriptions

```sql
CREATE TABLE IF NOT EXISTS prescriptions (
    prescription_id BIGINT,
    visit_id        BIGINT,
    patient_id      BIGINT,
    drug_name       STRING,
    drug_code       STRING,
    dosage          STRING,
    prescribed_date DATE,
    duration_days   INT,
    prescribing_doc INT
)
STORED AS ORC
TBLPROPERTIES ('orc.compress'='SNAPPY');
```

Table 4: Diagnoses (with Array and Map Data Types)

```sql
CREATE TABLE IF NOT EXISTS diagnoses (
    diagnosis_id    BIGINT,
    visit_id        BIGINT,
    patient_id      BIGINT,
    icd10_codes     ARRAY<STRING>,   -- Multiple diagnoses per visit
    symptoms        MAP<STRING, INT>, -- symptom name → severity score
```

```
    diagnosis_date  DATE,
    diagnosis_notes STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.ql.io.orc.OrcSerde'
STORED AS ORC;
```

Table 5: Hospitals

```
CREATE TABLE IF NOT EXISTS hospitals (
    hospital_id     INT,
    hospital_name   STRING,
    city            STRING,
    state           STRING,
    bed_count       INT,
    accreditation   STRING
)
STORED AS ORC;
```

Views

View 1: Patient Visit Summary

```
CREATE VIEW patient_visit_summary AS
SELECT
    p.patient_id,
    CONCAT(p.first_name, ' ', p.last_name)  AS patient_name,
    p.city,
    COUNT(v.visit_id)                       AS total_visits,
    SUM(v.bill_amount)                      AS total_spend,
    AVG(v.bill_amount)                      AS avg_bill,
    MAX(v.admission_date)                   AS last_visit_date,
    SUM(CASE WHEN v.readmitted = TRUE THEN 1 ELSE 0 END) AS
readmission_count
FROM patients p
JOIN hospital_visits v ON p.patient_id = v.patient_id
GROUP BY p.patient_id, p.first_name, p.last_name, p.city;
```

View 2: High-Risk Patients (Readmission Candidates)

```
CREATE VIEW high_risk_patients AS
SELECT
    patient_id,
    patient_name,
```

```
    total_visits,
    readmission_count,
    ROUND((readmission_count / total_visits) * 100, 2) AS
readmission_rate_pct
FROM patient_visit_summary
WHERE readmission_count >= 2
AND total_visits >= 3;
```

Sample Analytical Queries

Query 1: Top 10 Most Prescribed Drugs This Year

```
SELECT
    drug_name,
    COUNT(*)                AS prescription_count,
    COUNT(DISTINCT patient_id) AS unique_patients
FROM prescriptions
WHERE prescribed_date >= DATE_SUB(CURRENT_DATE, 365)
GROUP BY drug_name
ORDER BY prescription_count DESC
LIMIT 10;
```

Query 2: Monthly Hospital Admission Trend

```
SELECT
    visit_year,
    visit_month,
    COUNT(visit_id)         AS total_admissions,
    AVG(bill_amount)        AS avg_billing,
    SUM(CASE WHEN severity = 'Critical' THEN 1 ELSE 0 END) AS
critical_cases
FROM hospital_visits
GROUP BY visit_year, visit_month
ORDER BY visit_year, visit_month;
```

Query 3: Average Hospital Stay Duration by Diagnosis

```
SELECT
    diagnosis_code,
    COUNT(*)                                            AS case_count,
    ROUND(AVG(DATEDIFF(discharge_date, admission_date)), 1) AS
avg_stay_days
FROM hospital_visits
WHERE discharge_date IS NOT NULL
```

```
GROUP BY diagnosis_code
HAVING case_count > 50
ORDER BY avg_stay_days DESC;
```

Partitioning and Bucketing Strategy

- **Partitioning by year/month**: Most queries filter by time range; partitioning eliminates full table scans.
- **Bucketing by patient_id**: Enables efficient joins between `hospital_visits` and `prescriptions` using **Bucketed Map Join**, significantly speeding up patient-level analysis.
- **ORC + Snappy**: ORC (Optimized Row Columnar) format enables column pruning and predicate pushdown; Snappy compression reduces storage by ~60%.

Built-in Functions Used

Function	Usage
`COUNT`, `SUM`, `AVG`, `MAX`	Aggregations
`CONCAT`	String concatenation for patient name
`ROUND`	Rounding decimal values
`DATEDIFF`	Calculate hospital stay duration
`DATE_SUB`	Filter records from the past year
`CASE WHEN`	Conditional aggregation (critical cases, readmissions)
`CURRENT_DATE`	Dynamic date filtering

Hive–Hadoop Integration

- All Hive tables are stored as **ORC files in HDFS** under `/user/hive/warehouse/`.
- HiveQL queries are compiled to **Tez DAGs** (or MapReduce jobs) and submitted to **YARN** for distributed execution.
- The **Hive Metastore** (backed by MySQL) stores schema information independently of HDFS, allowing tools like **Spark SQL**, **Presto**, and **Impala** to reuse the same schema.
- **Hive LLAP (Live Long and Process)** can be enabled for sub-second interactive queries on this healthcare data.

Conclusion

The proposed Hive schema for HealthInsight Analytics leverages partitioning, bucketing, ORC storage, and rich HiveQL features to enable efficient, scalable analytical queries on petabyte-scale patient data stored in HDFS. The design balances query performance, storage efficiency, and analytical expressiveness required for a production healthcare analytics platform.

Module 2: Apache Spark and Spark SQL

Q1. (Understand – L2) Describe the key features of Apache Spark's architecture. Explain how it supports in-memory processing and differs from traditional disk-based systems like Hadoop MapReduce.

Introduction to Apache Spark

Apache Spark is an open-source, unified analytics engine for large-scale data processing. Originally developed at UC Berkeley's AMPLab in 2009, it was designed to overcome the performance limitations of Hadoop MapReduce by leveraging **in-memory computation**.

Spark provides a comprehensive suite of tools:

- **Spark Core** – Basic RDD-based distributed computing
- **Spark SQL** – Structured data queries via DataFrames/Datasets
- **Spark Streaming / Structured Streaming** – Real-time stream processing
- **MLlib** – Distributed machine learning
- **GraphX** – Graph processing

Key Features of Apache Spark

1. In-Memory Processing Spark stores intermediate computation results in RAM (as RDDs or DataFrames) rather than writing them to disk after every stage. This eliminates the disk I/O overhead that dominates MapReduce performance.

2. Lazy Evaluation Transformations (e.g., `map`, `filter`) are not executed immediately — they are recorded as a DAG. Execution happens only when an action (e.g., `count`, `collect`) is called, allowing the optimizer to plan efficient execution.

3. Unified Processing Engine A single Spark application can perform batch processing, stream processing, SQL queries, machine learning, and graph computations — eliminating the need for separate tools.

4. Rich APIs Spark supports **Scala**, **Python (PySpark)**, **Java**, and **R**, with high-level APIs for DataFrames, Datasets, and SQL — making it accessible to diverse users.

5. Fault Tolerance via Lineage Instead of replicating data across nodes, Spark tracks the **lineage** (sequence of transformations) of each RDD. If a partition is lost, Spark recomputes it from the lineage — efficient and lightweight.

6. Advanced DAG Scheduler Spark's DAG (Directed Acyclic Graph) scheduler optimizes the execution plan by:

- Pipelining narrow transformations (no shuffle needed) into a single stage
- Minimizing data shuffles across stages
- Reusing cached data across actions

7. Cluster Manager Agnostic Spark runs on **YARN** (Hadoop), **Apache Mesos**, **Kubernetes**, or its own **Standalone** cluster manager.

Spark Architecture

```
+-----------------------------------------+
|         Driver Program                  |
|  - SparkContext / SparkSession           |
|  - DAG Scheduler                        |
|  - Task Scheduler                       |
+-------------------+--------------------+
                    |
          +----------+-----------+
          |   Cluster Manager    |
          | (YARN / Mesos / K8s) |
          +-----+----------+-----+
                |          |
       +--------+--+    +--+--------+
       | Worker 1  |    | Worker 2  |
       | Executor  |    | Executor  |
       | - Cache   |    | - Cache   |
       | - Tasks   |    | - Tasks   |
       +-----------+    +-----------+
```

- **Driver**: Hosts the `SparkSession`, creates the DAG, schedules tasks.
- **Executor**: JVM process on each worker node. Runs tasks and stores cached data.
- **Cluster Manager**: Allocates resources (containers/VMs) for executors.
- **Task**: Smallest unit of work — runs on one partition of data in one executor.

Spark vs Hadoop MapReduce

Feature	Hadoop MapReduce	Apache Spark
Processing Model	Disk-based (write after every stage)	In-memory (RAM-first)
Speed	Baseline	10x–100x faster for iterative jobs
Ease of Use	Complex Java API	High-level Python/Scala APIs
Workloads	Batch only	Batch, Streaming, SQL, ML, Graph
Fault Tolerance	Data replication	Lineage-based recomputation
Latency	High (disk I/O)	Low (in-memory)
Shuffle	Disk-based, slow	Memory-optimized, faster
Real-time Support	No	Yes (Structured Streaming)
Intermediate Storage	HDFS	RAM / Disk (configurable)

In-Memory Processing: Deep Dive

In MapReduce, a 3-stage computation (Map → Sort → Reduce) looks like:

```
HDFS → Map → [write to disk] → Shuffle → [write to disk] → Reduce → HDFS
```

Each stage writes its entire output to HDFS before the next stage can begin.

In Spark:

```
HDFS → Stage 1 (pipelined: filter + map) → [optional persist to RAM] →
Stage 2 → output
```

- For iterative algorithms (e.g., k-means, PageRank), Spark **persists** the dataset in RAM after the first load. Subsequent iterations read from RAM — eliminating 10+ rounds of HDFS reads in MapReduce.
- Spark uses **LRU eviction** to spill data to disk only when RAM is full, ensuring graceful degradation.

Conclusion

Apache Spark's in-memory architecture, DAG-based optimization, lazy evaluation, and unified multi-workload support make it the dominant framework for large-scale data processing today. Its ability to perform iterative computations orders of magnitude faster than MapReduce has made it the de facto replacement in modern Big Data platforms.

Q2. (Apply – L3) Illustrate the lifecycle of RDDs in Spark, including their creation, transformations, and actions. Apply this to a scenario involving real-time data aggregation from IoT sensors deployed in a smart city traffic management system.

Introduction to RDDs

An **RDD (Resilient Distributed Dataset)** is the fundamental data abstraction in Apache Spark. It represents an **immutable**, **distributed** collection of objects that can be processed in parallel across a cluster.

Key Properties of RDDs:

- **Resilient**: Fault-tolerant via lineage graph — lost partitions can be recomputed
- **Distributed**: Data is partitioned and spread across cluster nodes
- **Dataset**: A collection of records of any type (integers, strings, tuples, objects)
- **Immutable**: Transformations produce new RDDs — original is never modified
- **Lazy**: Transformations are not computed until an action is called

RDD Lifecycle

```
Creation → Transformations (build DAG) → Action (triggers execution) →
Result
```

Phase 1: RDD Creation

RDDs can be created in three ways:

a) From external data sources:

```
sc = SparkContext("local", "TrafficApp")

# From HDFS
rdd = sc.textFile("hdfs:///traffic/sensor_data.csv")

# From local file
rdd = sc.textFile("file:///data/sensors.txt")
```

b) By parallelizing a collection:

```
data = [1, 2, 3, 4, 5]
rdd = sc.parallelize(data, numSlices=4)
```

c) By transforming an existing RDD:

```
filtered_rdd = rdd.filter(lambda x: x > 2)  # Creates a new RDD
```

Phase 2: Transformations

Transformations are **lazy** — they define the computation but do not execute it. Each transformation returns a new RDD.

Narrow Transformations (no data shuffle — each partition maps to one output partition):

- `map(func)` – Apply function to each element
- `filter(func)` – Keep elements that satisfy a condition
- `flatMap(func)` – Map then flatten results
- `mapPartitions(func)` – Apply function to each partition

Wide Transformations (require data shuffle across partitions):

- `groupByKey()` – Group values by key (expensive)
- `reduceByKey(func)` – Aggregate values per key (more efficient)
- `sortByKey()` – Sort by key
- `join(otherRDD)` – Join two RDDs by key

Phase 3: Actions

Actions **trigger execution** of the DAG and return a result to the driver or write to storage.

Action	Description
`count()`	Returns number of elements
`collect()`	Returns all elements to driver
`first()`	Returns first element
`take(n)`	Returns first n elements
`reduce(func)`	Aggregates all elements
`saveAsTextFile(path)`	Writes to HDFS/local
`foreach(func)`	Applies function to each element

Phase 4: Persistence / Caching

RDDs can be **cached in memory** to avoid recomputation across multiple actions:

```
rdd.persist(StorageLevel.MEMORY_AND_DISK)
# or shorthand:
rdd.cache()  # defaults to MEMORY_ONLY
```

Storage Levels:

- `MEMORY_ONLY` – Store in RAM as deserialized Java objects
- `MEMORY_AND_DISK` – Spill to disk if RAM is full
- `DISK_ONLY` – Store only on disk
- `MEMORY_ONLY_SER` – Serialize before storing (smaller footprint)

RDD Lineage (DAG)

Every RDD records its **lineage** — the sequence of transformations that created it. This enables **fault recovery without replication**.

```
textFile RDD
    ↓ map()
Parsed RDD
    ↓ filter()
Peak-hour RDD
    ↓ reduceByKey()
Junction Count RDD
    ↓ count() [ACTION → triggers execution]
```

If a partition of `Junction Count RDD` is lost, Spark replays the lineage from the `textFile RDD` for that partition only.

Real-World Scenario: Smart City Traffic Management

SmartFlow is a traffic management system with 10,000 IoT sensors deployed at intersections across a city. Each sensor emits records every second:

```
sensor_id, timestamp, junction_id, vehicle_count, avg_speed, vehicle_type
```

A Spark batch job aggregates the last hour of data every 5 minutes to:

1. Count vehicles per junction
2. Identify congestion hotspots (vehicle count > threshold)
3. Calculate average speed per junction
4. Save results for dashboard updates

```
from pyspark import SparkContext, SparkConf
from pyspark.storagelevel import StorageLevel

conf = SparkConf().setAppName("TrafficAnalysis")
sc = SparkContext(conf=conf)

# CREATION: Load last hour of sensor data from HDFS
raw_rdd = sc.textFile("hdfs:///traffic/data/last_hour/*.csv")

# TRANSFORMATION 1: Parse CSV into structured tuples
def parse_record(line):
    fields = line.split(",")
    return (fields[2], int(fields[4]), float(fields[5]))  # (junction_id,
vehicle_count, avg_speed)

parsed_rdd = raw_rdd.map(parse_record)

# TRANSFORMATION 2: Filter out corrupted records
valid_rdd = parsed_rdd.filter(lambda x: x[1] >= 0 and x[2] >= 0)

# CACHE: Reuse for multiple downstream actions
valid_rdd.persist(StorageLevel.MEMORY_AND_DISK)

# TRANSFORMATION 3: Prepare (junction_id, vehicle_count) pairs
vehicle_rdd = valid_rdd.map(lambda x: (x[0], x[1]))

# TRANSFORMATION 4: Sum vehicle counts per junction
junction_counts = vehicle_rdd.reduceByKey(lambda a, b: a + b)

# TRANSFORMATION 5: Identify congestion (vehicle_count > 500)
congestion_rdd = junction_counts.filter(lambda x: x[1] > 500)

# TRANSFORMATION 6: Average speed per junction
speed_rdd = valid_rdd.map(lambda x: (x[0], (x[2], 1))) \
                     .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1])) \
                     .mapValues(lambda x: round(x[0]/x[1], 2))
```

```
# ACTIONS: Trigger execution
print("Total sensor readings:", valid_rdd.count())                    #
Action 1
congestion_rdd.saveAsTextFile("hdfs:///traffic/output/congestion")   #
Action 2
speed_rdd.saveAsTextFile("hdfs:///traffic/output/avg_speed")         #
Action 3

sc.stop()
```

Execution Flow:

- `valid_rdd.count()` triggers DAG execution: read → parse → filter → count
- `congestion_rdd.saveAsTextFile()` reuses cached `valid_rdd` — no re-read from HDFS
- `speed_rdd.saveAsTextFile()` also reuses cached `valid_rdd`
- Without `.persist()`, each action would reload and reparse from HDFS — 3x slower

Conclusion

The RDD lifecycle — creation, lazy transformations, caching, and actions — gives Spark fine-grained control over distributed computation. In the smart city traffic scenario, RDD persistence enables efficient multi-step analysis on shared data, while the lineage graph provides robust fault tolerance without expensive data replication.

Q3. (Analyze – L4) Examine the differences between lazy evaluation and eager evaluation in Spark's transformation and action operations. Analyze how this affects resource utilization in distributed computing.

Introduction

Evaluation strategy determines when expressions in a program are computed. In distributed computing frameworks like Spark, this choice has profound implications for performance, memory usage, and fault recovery.

Eager Evaluation

In **eager evaluation**, every operation is executed **immediately** as it is called. The result is computed and stored before moving to the next statement.

Characteristics:

- Operations execute sequentially as they are encountered
- Results are immediately available after each operation
- No opportunity for global optimization
- Common in traditional imperative languages (Python lists, SQL with immediate execution)

Example (Eager — Python list):

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
filtered = [x for x in data if x % 2 == 0]    # Executes immediately
mapped = [x * x for x in filtered]             # Executes immediately
result = sum(mapped)                            # Executes immediately
# 3 full passes over the data
```

Lazy Evaluation in Spark

In **lazy evaluation**, transformations are **not executed** when called. Instead, Spark records them in a **DAG (Directed Acyclic Graph)** and executes the entire plan only when an **action** is invoked.

Characteristics:

- Transformations: `map`, `filter`, `flatMap`, `join`, `groupByKey`, etc. → **lazy**
- Actions: `count`, `collect`, `save`, `reduce`, `first` → **trigger execution**
- Spark's **Catalyst Optimizer** and **DAG Scheduler** analyze the full pipeline before executing
- Enables **pipelining** — multiple narrow transformations run in a single pass

Example (Lazy — Spark RDD):

```
rdd = sc.textFile("hdfs:///data/input.txt")    # Lazy: no file read yet
filtered = rdd.filter(lambda x: len(x) > 5)    # Lazy: DAG node added
mapped = filtered.map(lambda x: x.upper())     # Lazy: DAG node added
result = mapped.count()                         # ACTION: entire DAG
executes now
```

Lazy vs Eager Evaluation — Detailed Comparison

Aspect	Eager Evaluation	Lazy Evaluation (Spark)
Execution	Immediate on each operation	Deferred until action
Optimization	Per-operation only	Global DAG-wide optimization
Memory Usage	Stores all intermediate results	Pipelines intermediates, avoids storage
Short-circuit	Not possible	Possible (e.g., `first()` stops early)
Fault Recovery	Must recompute from scratch	Recompute only lost partition via lineage
Debugging	Easy — inspect after each step	Harder — error appears at action call
Throughput	Lower (redundant passes)	Higher (single-pass pipelining)

How Lazy Evaluation Optimizes Resource Utilization

1. Pipeline Narrow Transformations (Stage Fusion)

Without pipelining (eager):

```
READ → [store in RAM] → FILTER → [store in RAM] → MAP → [store in RAM] →
COUNT
```

3 intermediate datasets allocated.

With pipelining (lazy):

```
READ → FILTER → MAP → COUNT  (all in one pass per partition)
```

0 intermediate datasets stored — each record flows through the pipeline and is discarded.

2. Predicate Pushdown Spark's optimizer pushes `filter` operations as early as possible in the DAG — reading less data overall.

3. Short-Circuit Evaluation

```
result = rdd.filter(condition).first()
```

With lazy evaluation, once one matching element is found, Spark stops processing — no need to scan the entire dataset. Eager evaluation would process all elements first.

4. Avoid Redundant Computation

```
# Lazy — DAG is built but not executed
filtered_rdd = rdd.filter(lambda x: x > 0)
squared_rdd = filtered_rdd.map(lambda x: x**2)

# Only one action — DAG executes once
result = squared_rdd.reduce(lambda a, b: a + b)
```

If multiple actions are called on the same RDD without caching, the full lineage re-executes. This is a **trade-off of lazy evaluation** — mitigated by `.persist()`.

5. Memory Efficiency in Distributed Computing In a 1000-node cluster processing 1 TB of data:

- Eager: Each intermediate transformation materializes ~1 TB in distributed RAM — quickly exhausting memory.
- Lazy: Only one pass is made; intermediate data per record is a few bytes in CPU registers/local buffers.

Example: Impact on Resource Utilization

Scenario: Filter 1 TB dataset to 1 GB, then aggregate.

Eager (MapReduce style):

```
1. Read 1 TB from disk (1 TB I/O)
2. Write filtered 1 GB to disk (1 GB I/O)
3. Read 1 GB from disk (1 GB I/O)
4. Write aggregation result (small I/O)
Total disk I/O: ~2 TB
```

Lazy (Spark):

```
1. Read 1 TB from disk (1 TB I/O)
2. Filter in-memory, pipe to aggregation
3. Write final result (small I/O)
Total disk I/O: ~1 TB (50% reduction)
Memory peak: holds only one partition at a time
```

Spark's Execution Phases (Enabled by Lazy Evaluation)

```
Transformation calls → Build Logical Plan
                            ↓
                      Catalyst Optimizer (optimize logical plan)
                            ↓
                      Physical Plan (select execution strategy)
                            ↓
                      DAG Scheduler (split into stages)
                            ↓
                      Task Scheduler (assign tasks to executors)
                            ↓
                      Execution (only now does computation happen)
```

Conclusion

Lazy evaluation is one of Spark's most powerful architectural features. By deferring execution until actions are called, Spark can globally optimize computation pipelines, eliminate unnecessary data materialization, and minimize both disk I/O and memory consumption. In distributed environments processing terabytes of data, these optimizations translate directly into dramatically reduced resource costs and faster job completion times compared to eager execution models.

Q4. (Evaluate – L5) Critically evaluate the capabilities of Spark SQL for handling structured data queries. Assess its advantages over native SQL databases in big data environments.

Introduction to Spark SQL

Spark SQL is a Spark module for structured data processing. It provides a programming interface that lets you query structured data using either SQL syntax or the DataFrame/Dataset API. Under the hood, Spark

SQL uses the **Catalyst optimizer** and **Tungsten execution engine** to generate highly efficient execution plans.

Core Components of Spark SQL

```
+-----------------------------------------+
|    SQL / HiveQL / DataFrame / Dataset API |
+-----------------------------------------+
              ↓
+-----------------------------------------+
|        Catalyst Optimizer                |
|  - Analysis (resolve references)         |
|  - Logical Optimization (rule-based)     |
|  - Physical Planning (strategy selection)|
|  - Code Generation (Tungsten JIT)        |
+-----------------------------------------+
              ↓
+-----------------------------------------+
|       Tungsten Execution Engine          |
|  - Off-heap memory management            |
|  - Cache-aware data structures           |
|  - Whole-stage code generation           |
+-----------------------------------------+
              ↓
+-----------------------------------------+
|    Data Sources: HDFS, S3, Hive, JDBC,   |
|    Parquet, ORC, JSON, CSV, Avro, Delta  |
+-----------------------------------------+
```

Key Capabilities of Spark SQL

1. Unified Data Access Spark SQL can query data from multiple sources **in a single query**:

```python
df_parquet = spark.read.parquet("hdfs:///data/sales.parquet")
df_mysql = spark.read.jdbc(url, "customers", properties)
df_json = spark.read.json("s3://bucket/events.json")

# Join across all three sources
result = df_parquet.join(df_mysql, "customer_id") \
                   .join(df_json, "event_id")
result.show()
```

No traditional SQL database can federate across Parquet, JDBC, and S3 JSON in a single native query.

2. Catalyst Optimizer The Catalyst optimizer applies transformations to the logical plan:

- **Constant folding**: `WHERE 1=1 AND age > 25` → `WHERE age > 25`
- **Predicate pushdown**: Filter at the data source level (e.g., Parquet column pruning)

- **Column pruning**: Read only required columns from columnar formats
- **Join reordering**: Automatically reorder joins for efficiency
- **Subquery elimination**: Rewrite correlated subqueries

3. Tungsten Execution Engine

- **Off-heap memory**: Avoids JVM garbage collection overhead
- **Whole-stage code generation**: Compiles query plans into bytecode for near-native execution speed
- **Vectorized reads**: Reads columnar data in batches for CPU cache efficiency

4. Schema Enforcement and Type Safety

- **DataFrames**: Untyped at compile time (like SQL)
- **Datasets**: Typed at compile time (Scala/Java only) — catches errors early

5. Window Functions

```
SELECT
    employee_id,
    department,
    salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS
dept_rank,
    AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary
FROM employees;
```

6. Integration with MLlib and Streaming DataFrames created by Spark SQL can be directly passed to MLlib pipelines or Structured Streaming — eliminating data format conversions.

Critical Evaluation: Spark SQL vs Native SQL Databases

Advantages of Spark SQL over Native SQL Databases:

Dimension	Native SQL DB (PostgreSQL, MySQL)	Spark SQL
Scalability	Vertical scaling (bigger server)	Horizontal scaling (add nodes)
Data Volume	Practical limit: ~TB	No practical limit: PB+
Processing Model	Row-at-a-time (OLTP)	Vectorized batch (OLAP)
Fault Tolerance	WAL-based recovery	Lineage-based RDD recovery
Data Formats	Proprietary storage	Parquet, ORC, JSON, CSV, Avro, Delta
Multi-source joins	Limited (same DB or federated)	Any source in one query
Cost	Expensive licenses (Oracle, SQL Server)	Open-source, commodity hardware

Dimension	Native SQL DB (PostgreSQL, MySQL)	Spark SQL
Integration	Isolated	Native integration with ML, streaming, graph
Columnar Optimization	Limited	Full (Parquet + Catalyst pushdown)
Schema Flexibility	Strict schema required	Schema-on-read for semi-structured data

Limitations of Spark SQL vs Native SQL Databases:

Limitation	Details
OLTP Workloads	Not designed for high-frequency, low-latency single-row lookups
ACID Transactions	Limited natively (Delta Lake adds ACID support)
Startup Overhead	Spark cluster startup takes seconds–minutes; RDBMS responds in milliseconds
Indexing	No traditional B-tree indexes (relies on partitioning and column pruning)
Streaming Inserts	Not designed for millions of small inserts per second
Complexity	Higher operational complexity (cluster management, tuning)

Real-World Performance Comparison

Benchmark: TPC-DS (100 GB dataset)

Query Type	PostgreSQL	Spark SQL (10 nodes)
Simple SELECT with filter	0.5s	2s (cluster overhead)
Aggregation on 100GB	45 min	3 min
Multi-table join (5 tables)	20 min	2.5 min
Window functions on 100GB	Timeout	5 min

For analytical queries on large datasets, Spark SQL significantly outperforms traditional RDBMS.

When to Use Spark SQL vs RDBMS

Use Spark SQL	Use Native SQL DB
Dataset > 100 GB	Dataset < 100 GB
Batch analytics & reporting	OLTP (transactional operations)
Multi-source data federation	Single, well-structured data source
Integration with ML pipelines	Low-latency point queries

Use Spark SQL	Use Native SQL DB
Historical/archival data analysis	Real-time application data

Conclusion

Spark SQL is a powerful, enterprise-grade analytical query engine that excels in distributed big data environments. Its Catalyst optimizer, columnar format support, multi-source federation, and native integration with the broader Spark ecosystem give it significant advantages over traditional SQL databases for OLAP workloads at scale. However, it is not a replacement for RDBMS in transactional contexts — the two are complementary tools serving different purposes.

Q5. (Create – L6) Develop a high-level plan for implementing SQL queries in Spark to analyze a dataset of social media interactions, incorporating joins, filters, and aggregations for deriving user behavior insights.

Introduction

Social media platforms generate enormous volumes of interaction data — likes, comments, shares, follows, ad clicks, and video views. Analyzing this data at scale requires a distributed SQL-capable framework. Apache Spark SQL, with its DataFrame API and Catalyst optimizer, is ideally suited for this analytical use case.

This section presents a **complete, production-ready implementation plan** for a social media behavior analytics pipeline using Spark SQL.

Business Objectives

The analytics platform — **InsightPulse** — aims to derive:

1. **User Engagement Score**: Measure how actively a user interacts.
2. **Content Performance**: Identify top-performing posts and content types.
3. **Trending Hashtags**: Find hashtags gaining traction hourly.
4. **Churn Signals**: Identify users showing declining activity.
5. **Influencer Detection**: Find users with high reach and engagement ratios.

Phase 1: Data Sources and Schema

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.window import Window

spark = SparkSession.builder \
    .appName("InsightPulse") \
    .config("spark.sql.shuffle.partitions", "200") \
    .config("spark.sql.adaptive.enabled", "true") \
```

```
    .enableHiveSupport() \
    .getOrCreate()
```

Source Tables:

```
# Users table
users_df = spark.read.parquet("hdfs:///social/users/")
# Schema: user_id, username, signup_date, country, age_group, account_type

# Posts table
posts_df = spark.read.parquet("hdfs:///social/posts/")
# Schema: post_id, user_id, content_type, posted_at, hashtags (array),
caption

# Interactions table (partitioned by date)
interactions_df =
spark.read.parquet("hdfs:///social/interactions/date=2024-01-*/")
# Schema: interaction_id, user_id, post_id, interaction_type,
interacted_at, duration_seconds

# Follows table
follows_df = spark.read.parquet("hdfs:///social/follows/")
# Schema: follower_id, followee_id, followed_at

# Register as temp views for SQL usage
users_df.createOrReplaceTempView("users")
posts_df.createOrReplaceTempView("posts")
interactions_df.createOrReplaceTempView("interactions")
follows_df.createOrReplaceTempView("follows")
```

Phase 2: Data Preparation and Filtering

Step 1: Filter valid interactions (last 30 days, active users only)

```
filtered_interactions = spark.sql("""
    SELECT
        i.interaction_id,
        i.user_id,
        i.post_id,
        i.interaction_type,
        i.interacted_at,
        i.duration_seconds,
        u.country,
        u.age_group,
        u.account_type
    FROM interactions i
    JOIN users u ON i.user_id = u.user_id
    WHERE i.interacted_at >= DATE_SUB(CURRENT_DATE, 30)
```

```
      AND u.account_type != 'suspended'
      AND i.duration_seconds >= 0
""")

filtered_interactions.cache()  -- Cache for reuse across multiple queries
```

Phase 3: User Engagement Score

Engagement score = weighted sum of interactions per user.

```
engagement_score = spark.sql("""
    SELECT
        user_id,
        country,
        age_group,
        SUM(CASE
            WHEN interaction_type = 'like'    THEN 1
            WHEN interaction_type = 'comment' THEN 3
            WHEN interaction_type = 'share'   THEN 5
            WHEN interaction_type = 'view'    THEN 0.5
            WHEN interaction_type = 'save'    THEN 4
            ELSE 0
        END)                                     AS engagement_score,
        COUNT(DISTINCT post_id)                  AS
unique_posts_interacted,
        COUNT(*)                                 AS total_interactions,
        AVG(duration_seconds)                    AS avg_session_duration,
        COUNT(DISTINCT DATE(interacted_at))      AS active_days
    FROM filtered_interactions
    GROUP BY user_id, country, age_group
    ORDER BY engagement_score DESC
""")

engagement_score.createOrReplaceTempView("user_engagement")
```

Phase 4: Content Performance Analysis

```
content_performance = spark.sql("""
    SELECT
        p.post_id,
        p.user_id,
        p.content_type,
        p.posted_at,
        COUNT(*)                                  AS total_interactions,
        SUM(CASE WHEN i.interaction_type = 'like'    THEN 1 ELSE 0 END) AS
likes,
        SUM(CASE WHEN i.interaction_type = 'comment' THEN 1 ELSE 0 END) AS
```

```
comments,
        SUM(CASE WHEN i.interaction_type = 'share'   THEN 1 ELSE 0 END) AS
shares,
        AVG(i.duration_seconds)                   AS avg_view_duration,
        ROUND(
            (SUM(CASE WHEN i.interaction_type IN
('like','comment','share') THEN 1 ELSE 0 END)
             / NULLIF(COUNT(*), 0)) * 100, 2
        )                                         AS engagement_rate_pct
    FROM posts p
    JOIN interactions i ON p.post_id = i.post_id
    WHERE p.posted_at >= DATE_SUB(CURRENT_DATE, 30)
    GROUP BY p.post_id, p.user_id, p.content_type, p.posted_at
    HAVING total_interactions >= 10
    ORDER BY engagement_rate_pct DESC
""")

content_performance.createOrReplaceTempView("content_performance")
```

Phase 5: Trending Hashtags (Hourly)

```
trending_hashtags = spark.sql("""
    SELECT
        hashtag,
        DATE_TRUNC('hour', i.interacted_at)       AS hour_bucket,
        COUNT(*)                                   AS usage_count,
        COUNT(DISTINCT i.user_id)                  AS unique_users
    FROM posts p
    LATERAL VIEW EXPLODE(p.hashtags) ht AS hashtag
    JOIN interactions i ON p.post_id = i.post_id
    WHERE i.interacted_at >= DATE_SUB(CURRENT_DATE, 1)
    GROUP BY hashtag, DATE_TRUNC('hour', i.interacted_at)
""")

# Apply window function to rank hashtags per hour
window_spec =
Window.partitionBy("hour_bucket").orderBy(col("usage_count").desc())

trending_with_rank = trending_hashtags \
    .withColumn("rank", rank().over(window_spec)) \
    .filter(col("rank") <= 10)

trending_with_rank.createOrReplaceTempView("trending_hashtags")
```

Phase 6: Churn Detection (Declining Users)

```
churn_signals = spark.sql("""
    WITH monthly_activity AS (
        SELECT
            user_id,
            DATE_TRUNC('month', interacted_at)    AS activity_month,
            COUNT(*)                              AS monthly_interactions
        FROM interactions
        WHERE interacted_at >= ADD_MONTHS(CURRENT_DATE, -3)
        GROUP BY user_id, DATE_TRUNC('month', interacted_at)
    ),
    activity_trend AS (
        SELECT
            user_id,
            activity_month,
            monthly_interactions,
            LAG(monthly_interactions, 1) OVER (
                PARTITION BY user_id ORDER BY activity_month
            )                                     AS
prev_month_interactions
        FROM monthly_activity
    )
    SELECT
        user_id,
        activity_month,
        monthly_interactions,
        prev_month_interactions,
        ROUND(
            (monthly_interactions - prev_month_interactions)
            / NULLIF(prev_month_interactions, 0) * 100, 2
        )                                         AS pct_change
    FROM activity_trend
    WHERE pct_change < -50
      AND activity_month = DATE_TRUNC('month', CURRENT_DATE)
""")
```

Phase 7: Influencer Detection

```
influencer_analysis = spark.sql("""
    SELECT
        u.user_id,
        u.username,
        u.country,
        COUNT(DISTINCT f.follower_id)             AS follower_count,
        ue.engagement_score,
        cp.avg_engagement_rate,
        ROUND(ue.engagement_score
              / NULLIF(COUNT(DISTINCT f.follower_id), 0) * 1000, 4) AS
influence_index
    FROM users u
```

```
    JOIN follows f          ON u.user_id = f.followee_id
    JOIN user_engagement ue ON u.user_id = ue.user_id
    JOIN (
        SELECT user_id, AVG(engagement_rate_pct) AS avg_engagement_rate
        FROM content_performance
        GROUP BY user_id
    ) cp ON u.user_id = cp.user_id
    WHERE u.account_type = 'public'
    GROUP BY u.user_id, u.username, u.country, ue.engagement_score,
cp.avg_engagement_rate
    HAVING follower_count >= 1000
    ORDER BY influence_index DESC
    LIMIT 100
""")
```

Phase 8: Writing Results

```python
from datetime import datetime

run_date = datetime.now().strftime("%Y-%m-%d")

# Write to HDFS in Parquet format (partitioned)
engagement_score.write \
    .mode("overwrite") \
    .partitionBy("country") \
    .parquet(f"hdfs:///social/output/engagement/{run_date}/")

content_performance.write \
    .mode("overwrite") \
    .parquet(f"hdfs:///social/output/content_performance/{run_date}/")

trending_with_rank.write \
    .mode("overwrite") \
    .parquet(f"hdfs:///social/output/trending_hashtags/{run_date}/")

influencer_analysis.write \
    .mode("overwrite") \
    .parquet(f"hdfs:///social/output/influencers/{run_date}/")

spark.stop()
```

Phase 9: Performance Optimization Strategy

Technique	Applied Where
Partition pruning	`interactions` read with date filter
DataFrame caching	`filtered_interactions` reused across queries

Technique	Applied Where
Adaptive Query Execution	`spark.sql.adaptive.enabled = true` auto-optimizes joins
Broadcast join	Small `users` table broadcast to avoid shuffle
Columnar Parquet	All sources and outputs use Parquet for column pruning
Shuffle partition tuning	`spark.sql.shuffle.partitions = 200` (tuned for cluster)
LATERAL VIEW EXPLODE	Efficient hashtag expansion without UDF overhead
Window functions	Native Spark window avoids groupBy + join anti-pattern

---

Summary of Query Features Used

Feature	Queries
JOIN	Users–Interactions, Posts–Interactions, Users–Follows
FILTER (WHERE)	Date range, account type, interaction validity
AGGREGATION	COUNT, SUM, AVG, ROUND
CASE WHEN	Weighted engagement scoring
CTE (WITH clause)	Churn detection trend analysis
Window Functions	RANK(), LAG() for trending/churn
HAVING	Post-aggregation filtering
LATERAL VIEW EXPLODE	Array expansion for hashtags
NULLIF	Safe division to avoid divide-by-zero
DATE functions	DATE_SUB, DATE_TRUNC, ADD_MONTHS

Conclusion

This end-to-end Spark SQL implementation plan demonstrates how to architect a scalable social media analytics pipeline. By combining Spark SQL's expressive query capabilities (joins, window functions, CTEs, aggregations) with performance optimizations (caching, adaptive query execution, columnar storage), InsightPulse can derive rich user behavior insights from billions of interactions efficiently, cost-effectively, and reliably on a distributed cluster.